



UNIT-IV

CATALOGING AND INDEXING

User Search Techniques: Search Statements and Binding, Similarity Measures and Ranking, Relevance Feedback, Selective Dissemination of Information Search, Weighted Searches of Boolean Systems, Searching the INTERNET and Hypertext

Information Visualization: Introduction to Information Visualization, Cognition and Perception, Information Visualization Technologies

Search Statements

- Search statements are the elaboration of an information need. It is generated by "users" to specify the "concepts". i.e. the user may have ability to weight i.e. to different concepts in the search statements (traditional Boolean logic).
- The search statements directly affect the ability of information retrieval system to find the relevant items.
- The longer "search query" is easier to find those raw relevant items in the system.
- **BINDING:** It is based on the vocabulary and past experience of user.
 - Binding means when a more abstract form is specified into a more specific form.
 - In binding the search statement is passed to user, by a specific search system.
 - The search system translates the query to its own metalanguage.
 - In the final level of binding:
 - The Search is applied to "specific database".
 - This binding is based on the statistics of the processing tokens in the database.
 - Statistical based on the current contents of the database.
 - "Binding" is the process of translating or mapping the user's abstract request into specific terms and formats that a computer system can process. There are three levels of binding:
 - User-Level Binding – Concept to Words
 - System-Level Binding – Query to Search Engine Metalanguage
 - Database-Level Binding – Match Query with Data

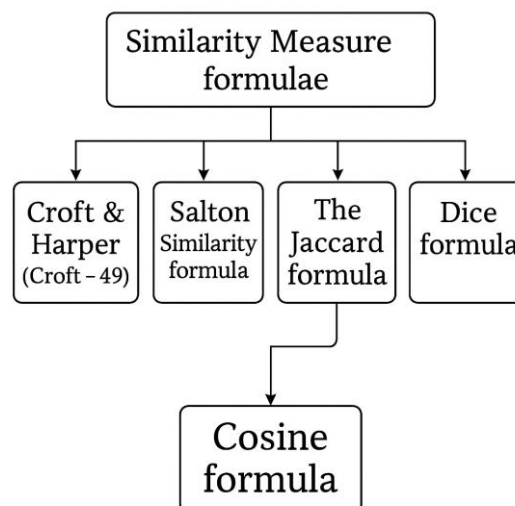
INPUT	Binding
"Find me information on the impact of the oil spills in Alaska on the price of oil"	User search statement using vocabulary of user
impact, oil (petroleum), spills (accidents), Alaska, price (cost, value)	Statistical system binding extracts processing tokens
impact (.308), oil (.606), petroleum (.65), spills (.12), accidents (.23), Alaska (.45), price (.16), cost (.25), value (.10)	Weights assigned to search terms based upon inverse document frequency algorithm and database

Figure 7.1 Examples of Query Binding

- "Find me information on the impact of oil spills in Alaska on the price of oil."
- Step 1 – User Binding:
 - Extracted terms: impact, oil, spills, Alaska, price, etc.
- Step 2 – System Binding:
 - Mapped to synonyms and given weights:
 - oil (.606), petroleum (.65) price (.16), cost (.25), value (.10) etc.
- Step 3 – Database Binding:
 - Weights adjusted again based on document statistics and indexing semantics of that particular database.

Similarity Measures and Ranking

- In general, searching is concerned with calculating the "similarity" between "a user's search statement" and "the items in the database".
- The similarity is applied to "total items" (or) logical passages in the item.
- For EXAMPLE, every paragraph may be defined as a passage (or) every 100 words.



• SIMILARITY MEASURES

- The different similarity measures can be used to calculate the similarity between “the item and the search statement”.
- The similarity between documents for clustering purposes, i.e.
- **Simple Sum of Products**

$$\text{SIM}(\text{Item}_i, \text{Item}_j) = \sum (\text{Term}_{i,k}) (\text{Term}_{j,k})$$

- The above formula uses the summation of the product of the various terms of two items.
- Calculates similarity by summing the product of corresponding term weights between two items.
- This is a basic approach but lacks normalization, which can lead to issues when items vary in length.

– **Croft's Similarity Formula**

$$\text{SIM}(\text{DOC}_i, \text{QUERY}_j) = \sum_{i=1}^Q (C + \text{IDF}_i) * f_{i,j}$$

$$f_{i,j} = K + (K - 1) \text{TF}_{i,j} / \text{maxfreq}_j$$

- C is a tuning constant.
- IDF_i is the inverse document frequency, which gives higher weight to rarer terms.
- where: K is a tuning constant (typically 0.3 to 0.5).
- $\text{TF}_{i,j}$ is the frequency of term i in item j.
- maxfreq_j is the maximum frequency of any term in item. This formula adjusts term weights based on their frequency and rarity, improving relevance.

– **Cosine Similarity**

$$\text{SIM}(\text{DOC}_i, \text{QUERY}_j) = \frac{\sum_{k=1}^n (\text{DOC}_{i,k} * \text{QTERM}_{j,k})}{\sqrt{\sum_{k=1}^n (\text{DOC}_{i,k})^2 * \sum_{k=1}^n (\text{QTERM}_{j,k})^2}}$$

- Measures the cosine of the angle between two vectors (document and query). A value of 1 means the vectors are identical (same direction), and 0 means they are orthogonal (unrelated).
 - The denominator normalizes for vector length, ensuring the result is between 0 and 1.
 - A variant (fourth equation) simplifies the denominator but still normalizes the score.
- ### – **Jaccard and Dice Measures**
- Jaccard: The denominator depends on the number of common terms, producing scores between -1 and 1. It penalizes dissimilarities more heavily.

$$\text{SIM}(\text{DOC}_i, \text{QUERY}_j) = \frac{\sum_{k=1}^n (\text{DOC}_{i,k} * \text{QTERM}_{j,k})}{\sum_{k=1}^n \text{DOC}_{i,k} + \sum_{k=1}^n \text{QTERM}_{j,k} - \sum_{k=1}^n (\text{DOC}_{i,k} * \text{QTERM}_{j,k})}$$

- Dice: The Dice measure simplifies the denominator and adds a factor of 2 in the numerator, also ranging from -1 to 1, but it's less sensitive to the number of common terms.

$$\text{SIM}(\text{DOC}_i, \text{QUERY}_j) = \frac{2 * \sum_{k=1}^n (\text{DOC}_{i,k} * \text{QTERM}_{j,k})}{\sum_{k=1}^n \text{DOC}_{i,k} + \sum_{k=1}^n \text{QTERM}_{j,k}}$$

- The simple “sum of the products” similarity formula is used to calculate similarity between the query and each document. If no threshold is specified, all three documents are considered hits. If a threshold of 4 is selected, then only DOC₁ is returned.

	QUERY = (2, 2, 0, 0, 4) DOC1 = (0, 2, 6, 4, 0) DOC2 = (2, 6, 0, 0, 4)		
	Cosine	Jaccard	Dice
DOC1	36.66	16	20
DOC2	36.66	-12	20

Figure 7.2 Normalizing Factors for Similarity Measures

- Query threshold process:
- Threshold defines the items in the relevant + little from the query.
- If no threshold is specified, all the documents are considered here.

Vector:	American, geography, lake, Mexico, painter, oil, reserve, subject
DOC1	geography of Mexico suggests oil reserves are available vector (0, 1, 0, 2, 0, 3, 1, 0)
DOC2	American geography has lakes available everywhere vector (1, 3, 2, 0, 0, 0, 0, 0)
DOC3	painters suggest Mexico lakes as subjects vector (0, 0, 1, 3, 3, 0, 0, 2)
QUERY	oil reserves in Mexico vector (0, 0, 0, 1, 0, 1, 1, 0)

$$\text{SIM}(\text{Q}, \text{DOC1}) = 6, \text{SIM}(\text{Q}, \text{DOC2}) = 0, \text{SIM}(\text{Q}, \text{DOC3}) = 3$$

Figure 7.3 Query Threshold Process

$$\begin{aligned}\text{SIM}(Q, \text{DOC}_1) &= 0 \times 0 + 0 \times 1 + 0 \times 1 + 0 \times 0 + 1 \times 2 + 0 \times 0 + 0 \times 0 + 1 \times 3 + 1 \times 1 + 0 \times 0 \\ \text{SIM}(Q, \text{DOC}_1) &= 2 + 3 + 1 \\ \text{SIM}(Q, \text{DOC}_1) &= 6\end{aligned}$$

$$\begin{aligned}\text{SIM}(Q, \text{DOC}_2) &= 0 \times 1 + 0 \times 3 + 0 \times 3 + 0 \times 2 + 1 \times 0 + 0 \times 1 + 0 \times 0 + 1 \times 0 + 1 \times 0 + 0 \times 0 \\ \text{SIM}(Q, \text{DOC}_2) &= 0\end{aligned}$$

$$\begin{aligned}\text{SIM}(Q, \text{DOC}_3) &= 0 \times 0 + 0 \times 0 + 0 \times 1 + 1 \times 3 + 0 \times 3 + 1 \times 3 + 0 \times 0 + 1 \times 0 + 1 \times 0 + 0 \times 2 \\ \text{SIM}(Q, \text{DOC}_3) &= 3\end{aligned}$$

- Ranking:
 - Once items are identified as possibly relevant to the user's query, it is the best way to present the most likely relevant items first.
 - This process is called "Ranking".

Relevance Feedback

- Relevance feedback concept was that, the new query should be based on the "old query".
- The old query modified to increase the weight of terms in "relevant items" and decrease the weight of terms that are in "non-relevant items".
- The first major work on relevance feedback was published in 1965 by Rocchio.
- This technique not only modified the terms in the original query but also allowed expansion of new terms from the relevant items.
- The revised Rocchio formula for query modification:

where

Q_n = the revised vector for the new query
 Q_o = the original query
 r = number of relevant items
 DR_i = the vectors for the relevant items
 nr = number of non-relevant items
 DNR_j = the vectors for the non-relevant items.

$$Q_n = Q_o + \frac{1}{r} \sum_{i=1}^r DR_i - \frac{1}{nr} \sum_{j=1}^{nr} DNR_j$$

$$Q_n = \alpha Q_o + \beta \sum_{i=1}^r DR_i - \gamma \sum_{j=1}^{nr} DNR_j$$

1. **Original Query** (αQ_o): Retains the original query terms, scaled by α (usually close to 1) to ensure the user's initial intent is preserved.
2. **Positive Feedback** ($\beta \frac{1}{r} \sum_{i=1}^r DR_i$): Adds the average vector of relevant documents, scaled by β . This increases the weight of terms that appear in relevant documents, pulling the query closer to these documents in the vector space.
3. **Negative Feedback** ($\gamma \frac{1}{nr} \sum_{j=1}^{nr} DNR_j$): Subtracts the average vector of non-relevant documents, scaled by γ . This decreases the weight of terms associated with non-relevant documents, pushing the query away from them.

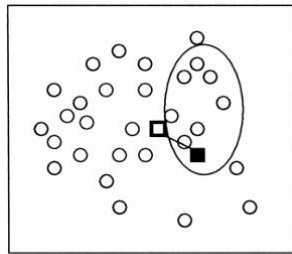
Constants:

- β (positive feedback) is typically weighted more heavily than γ (negative feedback) because positive feedback is more reliable for improving relevance.
- In practice, γ is often set to a small value or even 0 (ignoring negative feedback), as negative feedback can sometimes harm performance by moving the query away from potentially relevant items.

Positive Feedback

Positive feedback is weighted and significantly greater than negative feedback.

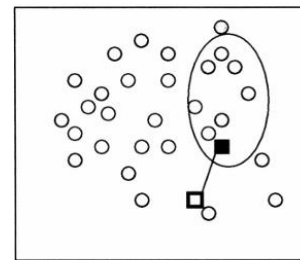
- Many times the positive feedback is used in relevance environment.
- Positive feedback is more likely to move a query closer to user's information need.



Positive Feedback

Negative feedback

- Negative feedback may help, but in some cases it actually reduces the effectiveness of a query.



Negative Feedback

- This "Impact of relevance feedback" figure visually shows how positive and negative feedback affect the query's position in the document space:
- Circles: Represent documents (filled circles are non-relevant, open circles are relevant).
- Oval: The set of items retrieved by the query.
- Solid Box: The original query's position.
- Hollow Box: The query's position after feedback.

Example:

$$Q_n = (3, 0, 0, 2, 0) + \frac{1}{4} (2+1, 4+3, 0+0, 0+0, 2+0) - \frac{1}{4} (0, 0, 4, 3, 2) \\ = (3\frac{3}{4}, 1\frac{3}{4}, 0, 1\frac{3}{4}, 0)$$

	Term 1	Term 2	Term 3	Term 4	Term 5
Q_o	3	0	0	2	0
$DOC1_r$	2	4	0	0	2
$DOC2_r$	1	3	0	0	0
$DOC3_{nr}$	0	0	4	3	3
Q_n	$3\frac{3}{4}$	$1\frac{3}{4}$	0	$1\frac{3}{4}$	0

Figure 7.7 Query Modification via Relevance Feedback

$$\text{Constants: } \alpha = 1, \beta = \frac{1}{2} \times \frac{1}{2} = \frac{1}{4} \text{ (since } r = 2), \gamma = \frac{1}{4} \times 1 = \frac{1}{4} \text{ (since } nr = 1).$$

- Recent experiments with relevance feedback during TREC sessions have shown conclusively the advantages of relevance feedback.
- Queries using relevance feedback produce significantly better results than queries being manually enhanced, while user enter queries with few no. of terms, automatic relevance feedback based on the rank value of that items used.
- This concept in the information system called pseudo-relevance feedback, blind feedback (or local text analysis). It does not require human relevance feedback.
- Highest ranked items from query are automatically assumed to be relevant.

Selective Dissemination of Information Search

- Selective dissemination of information frequently called “dissemination systems” are becoming more prevalent with growth of the internet.
- A dissemination system sometimes labeled as a “push system” and whereas a search system called a “pull system”.
- SDI systems, also called dissemination systems, automatically deliver new items to users when they match a user’s profile (a static query representing the user’s interests).
- Key characteristics include:
 - Push vs. Pull: In SDI, the system pushes relevant items to the user as new data arrives, unlike search systems where users pull information via queries.
 - Profiles: These are static, broad search statements with many terms (hundreds, compared to a few in ad hoc queries), covering a user’s general information needs.
 - Time Parameter: Profiles are active as long as their time parameter is in the future, delivering items asynchronously to the user’s "mail" file.
- Examples
 - Logicon Message Dissemination System (LMDS):
 - Treats profiles as a static database and the new item as a query.
 - Uses a least frequently occurring trigraph (three-character sequence) algorithm to quickly eliminate profiles that don’t match the item, then analyzes potential matches in detail.
 - Personal Library Software (PLS):
 - Accumulates new items into a database and periodically runs profiles against it, sacrificing real-time delivery for the use of retrospective search software.
 - RetrievalWare and InRoute:
 - RetrievalWare: Uses a statistical algorithm that doesn’t rely on corpora data, comparing each profile to the item after filtering out irrelevant terms.
 - InRoute: Uses IDF, which it computes and updates as items arrive, stabilizing over time as more items are processed.

Weighted Searches of Boolean Systems

- Boolean queries (e.g., "A AND B", "A OR B", "A NOT B") are traditionally strict—they return items that exactly match the conditions (e.g., both A and B for AND).

- In weighted index systems (where terms have weights representing their importance) strict Boolean logic can be too restrictive (AND) or too general (OR), leading to suboptimal retrieval results.
- Additionally, pure Boolean systems lack ranking, which is critical for user experience in information retrieval.
- The goal is to combine Boolean logic with weighted systems to allow for fuzziness and ranking.

1. Fuzzy Set Approach (Fox and Sharat)

- Fuzzy sets introduce the concept of "degree of membership" (DEG), allowing terms to partially belong to a set rather than strictly being in or out. This is useful for combining Boolean logic with weights.
- Degree of Membership for AND and OR

$$DEG_{A \cap B} = \min(DEG_A, DEG_B)$$

$$DEG_{A \cup B} = \max(DEG_A, DEG_B)$$

2. Mixed Min and Max (MMM) Model

- The MMM model calculates the similarity between a query (QUERY) and a document (DOC) as a linear combination of minimum and maximum weights of the terms:

$$\begin{aligned} SIM(QUERY_{OR}, DOC) &= C_{OR1} * \max(DOC_1, DOC_2, \dots, DOC_n) + \\ &\quad C_{OR2} * \min(DOC_1, DOC_2, \dots, DOC_n) \\ SIM(QUERY_{AND}, DOC) &= C_{AND1} * \min(DOC_1, DOC_2, \dots, DOC_n) + \\ &\quad C_{AND2} * \max(DOC_1, DOC_2, \dots, DOC_n) \end{aligned}$$

Here, C_{OR1} and C_{OR2} are weighting coefficients (best performance when C_{OR1} is 0.5 to 0.8 and $C_{OR2} > 0.2$).

3. Paice's Extension of MMM

- Paice expanded the MMM model by considering all term weights (not just min/max) and sorting them:
- Similarity is calculated as:

$$SIM(QUERY, DOC) = \sum_{i=1}^n r^{i-1} d_i / \sum_{i=1}^n r^{i-1}$$

d_i : Term weights, sorted in ascending order for AND queries, descending for OR queries.

r : Weighting coefficient (best as 1.0 for AND, 0.7 for OR).

- This method requires more computation due to sorting but provides a more comprehensive use of weights.

4. P-norm Model

- For OR queries, the origin (all weights = 0) is the worst case; the best documents are farthest from the origin.
- For AND queries, the ideal point is the unit vector (all weights = 1); the best documents are closest to this point.

$$Q_{OR} = (A_1, a_1) \text{ OR } (A_2, a_2) \text{ OR } \dots \text{ OR } (A_n, a_n)$$

$$Q_{AND} = (A_1, a_1) \text{ AND } (A_2, a_2) \text{ AND } \dots \text{ AND } (A_n, a_n)$$

- Similarity formulas:

$$\text{SIM}(Q_{\text{OR}}, \text{DOC}) = \sqrt[n]{(a_1^S d_{A1}^S + \dots + a_n^S d_{An}^S) / (a_1^S + a_2^S + \dots + a_n^S)}$$

$$\text{SIM}(Q_{\text{AND}}, \text{DOC}) = 1 - \sqrt[n]{(a_1^S (1 - d_{A1}^S) + \dots + a_n^S (1 - d_{An}^S)) / (a_1^S + a_2^S + \dots + a_n^S)}$$

$$\text{SIM}(Q_{\text{NOT}}, \text{DOC}) = 1 - \text{SIM}(Q, \text{DOC})$$

5. Salton's Approach (No Weights in Indexes)

- Salton's method applies Boolean operations first, then refines results using weights.

Boolean Operations

"A OR B" retrieves those items that contain the term A or the term B or both

"A AND B" retrieves those items that contain both terms A and B

"A NOT B" retrieves those items that contain term A and not contain term B.

Weighted Interpretation

"A₁ OR B₀" should return the set of items that contain A as a term

"A₁ AND B₀" will also return the set of items that contain term A

"A₁ NOT B₀" also return set A.

Two circles represent sets A and B.

Areas:

White: (A NOT B)

Black dotted: (A AND B)

Grey: (B NOT A)

For OR:

At weight 0.0 for B, only A (white + black dotted) is included.

As B's weight increases to 1.0, grey area (B NOT A) is added proportionally.

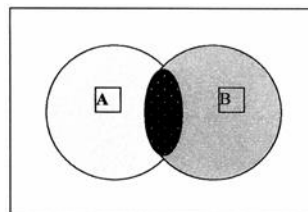


Figure 7.10 VENN Diagram

For AND:

At weight 1.0 for B, only black dotted (A AND B) is included.

As B's weight decreases to 0.0, white area (A NOT B) is added.

For NOT:

At weight 0.0 for B, all of A (white + black dotted) is included.

As B's weight increases to 1.0, black dotted (A AND B) is removed, leaving grey.

Example:

	Computer	program	cost	sale
D1	0	4	0	8
D2	0	2	0	0
D3	4	0	2	4
D4	0	6	4	6
D5	0	4	6	4
D6	6	0	4	0
D7	0	0	0	0
D8	4	2	0	2

$$Q1 = \text{QUERY}_1 = \text{Computer}_{1,0} \text{ OR } \text{program}_{0,333}$$

$$Q2 = \text{QUERY}_2 = \text{cost}_{75} \text{ AND } \text{sale}_{1,0}$$

$$Q1_{\text{strict interpretation}} = (D1, D2, D3, D4, D5, D6, D8)$$

$$Q2_{\text{strict interpretation}} = (D3, D4, D5)$$

$$Q1_{\text{invariant}} = (D8)$$

$$Q2_{\text{invariant}} = (D3, D4, D5)$$

$$Q1_{\text{optional}} = (D1, D2, D3, D4, D5, D6) \text{ thus } \lceil .333 \text{ times } 6 \text{ items} \rceil = 2 \text{ items}$$

$$Q2_{\text{optional}} = (D1, D8) \text{ which means } \lceil (1 - .75) \text{ times } 2 \text{ items} \rceil = 1 \text{ item}$$

Figure 7.11 Example of Weighted Boolean Query

Step-by-Step for Q1:

1. Strict Interpretation ($\text{computer}_1 \text{ OR } \text{program}_{0.333}$):

- Items with "computer" or "program": D1, D2, D3, D4, D5, D6, D8.
- $Q1_{\text{strict interpretation}} = \{D1, D2, D3, D4, D5, D6, D8\}$.

2. Invariant Set (program weight = 0):

- If "program" weight is 0, only "computer" matters: D3, D6, D8.
- $Q1_{\text{invariant}} = \{D3, D6, D8\}$.

3. Centroid of Invariant Set:

- D3: (4, 0, 2, 4), D6: (6, 0, 4, 0), D8: (4, 2, 0, 2)
- Centroid: $(4 + 6 + 4)/3, (0 + 0 + 2)/3, (2 + 4 + 0)/3, (4 + 0 + 2)/3 = (4.67, 0.67, 2, 2)$
- CENTROID($Q1$) = (4.2, 0.2, 2).

4. Number of Items to Add:

- Non-invariant set: D1, D2, D4, D5 (4 items).
- Program weight = 0.333.
- Items to add: $\lceil 0.333 \times 4 \rceil = \lceil 1.332 \rceil = 2$.

5. Similarity to Centroid:

- D1: (0, 4, 0, 8) $\rightarrow \text{SIM} = 0 + 8 + 0 + 24 = 24$
- D2: (0, 2, 0, 0) $\rightarrow \text{SIM} = 0 + 0 + 0 + 4 = 4$
- D4: (0, 6, 4, 6) $\rightarrow \text{SIM} = 0 + 12 + 0 + 24 = 36$
- D5: (0, 4, 6, 4) $\rightarrow \text{SIM} = 0 + 8 + 0 + 16 = 24$
- Sorted (descending): D4 (36), D1 (24), D5 (24), D2 (4).
- Add top 2: D4, D1 (D5 tied with D1, but D1 is chosen based on order).

Ranking Algorithms

1. Coarse Grain Ranking
2. Fine Grain Ranking

Ranking Algorithms - Coarse Grain Ranking

Completeness:	Semantic Distance	Contextual Evidence:
Example: If the query has 5 terms and the item contains 3 of them (or their synonyms), the completeness is $3/5 = 0.6$. This sets an upper limit on the item's rank. If query terms are weighted (e.g., some terms are marked as more important), those weights are factored into the score.	Synonyms (e.g., "buy" for "purchase") increase the score, while antonyms (e.g., "sell" for "buy") decrease it. The closer the semantic relationship, the more weight is added to the ranking.	Example: If the query term is "charge" with the context of "paying for an object," finding words like "buy," "purchase," or "debt" in the item suggests that "charge" is used in the desired sense, increasing the item's score. This helps disambiguate terms with multiple meanings (e.g., "charge" as in payment vs. "charge" as in an electrical charge).

Ranking Algorithms - Fine Grain Ranking

Proximity	Impact of Proximity
- A proximity factor is calculated, which is highest when terms are adjacent and decreases as the distance between terms increases. - Example: If the query terms "machine" and "learning" appear next to each other (e.g., "machine learning"), the item scores higher than if they are in different paragraphs (e.g., "machine" in the introduction and "learning" in the conclusion).	- In long documents, if query terms are widely distributed (e.g., one term at the beginning and another at the end), the fine grain rank can drop significantly, even to zero, despite the terms being present. - This reflects the intuition that scattered terms are less likely to indicate relevance to the query as a whole.



Searching the INTERNET and Hypertext

- Internet Search Mechanisms

The Internet in the 1990s relied on search engines like Yahoo, AltaVista, and Lycos to help users find information. These search engines worked by:

- Indexing Process: Automated processes (called crawlers or spiders) visited websites, retrieved text, and created indexes. These indexes stored references to web content, including URLs (web addresses), to help users locate relevant pages.
 - Lycos indexed only the home pages of websites, focusing on a limited scope.
 - AltaVista indexed all text on a website, providing a more comprehensive search.
- Ranking Algorithms: Search results were ranked using simple statistical methods based on word frequency in the indexed text. This helped prioritize results most relevant to the user's query.

- **Six Key Characteristics of Intelligent Agents**

Intelligent agents were software programs designed to autonomously search the Internet for information based on user-defined needs.

- 1. Autonomy**

- Agents operated independently without constant human input, navigating websites based on predefined criteria to collect relevant information.

- 2. Communication Ability**

- Agents used standard protocols (e.g., Z39.50, a library search protocol) to interact with websites and retrieve data.

- 3. Capacity for Cooperation**

- Agents could work together or with other systems to achieve common goals, such as sharing search results.

- 4. Capacity for Reasoning**

- Rule-based: Followed user-defined conditions and actions (e.g., "if a page contains 'AI,' save it").
- Knowledge-based: Learned from past actions to improve future searches.
- Artificial Evolution-based: Created new, smarter agents to handle complex tasks.

- 5. Adaptive Behavior**

- Agents assessed their environment and adjusted their actions to better meet user needs, combining autonomy and reasoning.

- 6. Trustworthiness**

- Users needed to trust that agents would act in their best interests, retrieving relevant and accessible information.

Information Visualization

- Doyle (1962): Proposed "semantic road maps" to give users a visual overview of a database's content, allowing them to focus queries on specific themes.
- Miller (1968): Emphasized spatial organization to map database information.
- Sammon (1969): Developed a non-linear mapping algorithm to reveal document associations, enabling visual "road maps."
- By the 1990s, technological advancements and the explosion of digital information made visualization a practical and commercially viable field.
- **Why Information Visualization Matters ?**
 - Historically, IR systems have focused on algorithmic improvements (e.g., indexing, searching, clustering) rather than how information is displayed to users. This was due to:
 - Technological limitations: Early systems lacked the capability for sophisticated visual displays.
 - Academic focus: Researchers prioritized algorithmic challenges over human-computer interface (HCI) design.
 - Multidisciplinary complexity: Effective visualization requires integrating cognitive science, perception, and HCI.
- Visualization can be divided into:
 - Link visualization: Shows relationships between items (e.g., networks or connections).
 - Attribute visualization: Reveals content patterns across many items (e.g., how search terms influence results).

Cognition and Perception

- The shift in user-machine interfaces from basic typewriter-like interactions to more complex systems like WIMP (windows, icons, menus, pointer) interfaces, which handle multiple tasks simultaneously.
- As computer displays became common, the focus turned to representing information visually in ways that align with human cognitive processes.
- The goal is to reduce the mental effort (cognitive overhead) users spend finding and understanding information by leveraging human perception—particularly vision, but also other senses like audio and touch.
- **Background on Vision and Cognition**
 - **Gestalt Psychology:** The mind organizes visual input into meaningful wholes using rules:
 - **Proximity:** Nearby objects are grouped together.
 - **Similarity:** Similar objects are grouped together.
 - **Continuity:** Smooth, continuous patterns are preferred (e.g., a circle with a line through it is seen as a circle and a line, not two half-circles).
 - **Closure:** Gaps are mentally filled to form a whole (e.g., a dashed square is still perceived as a square).
 - **Connectedness:** Linked elements are seen as a single unit.

• Aspects of the Visualization Process

○ Preattentive Processing

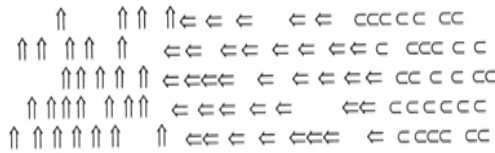


Figure 8.1 Preattentive Detection Mechanism

- The figure shows three groups of shapes: upward arrows (left), left-right arrows (middle), and "C" shapes (right).
- The visual system quickly detects the boundary between the upward and left-right arrows due to orientation differences (preattentive).

- Detecting the boundary between left-right arrows and "C" shapes requires more conscious effort because it involves different objects, not just orientations.
- Preattentive processes are automatic, low-level, and preconscious. They detect basic visual features like borders or orientation changes quickly.
- Implication: Information encoded in orientations (e.g., grouping by direction) is easier to detect than using different shapes, as it leverages the retina's feature detectors.

○ Rotated Objects and Symmetry

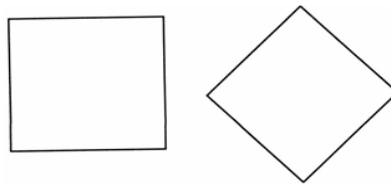


Figure 8.2 Rotating a Square and Reversing Letters in "REAL"

- The figure shows a square (left) and a rotated square (right, now a diamond).
- It also shows the word "REAL" with letters reversed.
- The rotated square is harder to recognize as a square, and reversed letters are harder to read because they deviate from familiar orientations.

- Recognizing rotated objects (e.g., a square turned 45 degrees) requires more cognitive effort than upright ones, as symmetry is easier to detect along vertical axes.
- Figure 8.2 (Rotating a Square and Reversing Letters in "REAL"):
- Implication: Interfaces should use familiar orientations (e.g., vertical/horizontal) to reduce cognitive load.

- Optical Illusions and Color
 - A light object on a dark background appears larger than a dark object on a light background. Small objects in displays should use bright colors to stand out.
 - **Color Use:**
 - Colors (hue, saturation, lightness) are used to organize and enhance visuals. Humans are drawn to primary colors (red, blue, green, yellow) and retain them longer.
 - **Depth Cues:**
 - Monocular cues like shading, perspective, and occlusion depict depth. Brightness enhances depth perception more than contrast.
 - Depth/size recognition is learned early (Gibson and Walk, 1960, showed six-month-olds understand depth), making depth a reliable way to represent information.
- Configural Effects

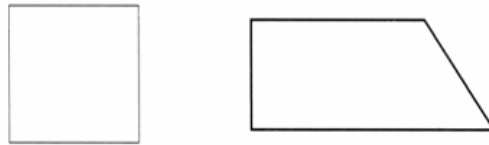


Figure 8.3 Distortions of a Regular Polygon

- Figure 8.3 (Distortion of a Regular Polygon): The figure shows a square (left) and a distorted square (right) with one side elongated.
- The visual system quickly detects the distortion because it deviates from the expected equal-sided shape.

- Configural clues allow quick recognition of abstract conditions by arranging objects in recognizable patterns.
- **Implication:** Configural clues can substitute high-level cognitive processes with faster low-level visual ones, useful for monitoring systems (e.g., detecting anomalies in operations).

- Spatial Frequency
 - The visual system constructs images from multiple channels (spatial frequency, orientation, contrast).
 - Spatial frequency measures light-dark cycles per degree of visual field.
 - Distinct images are easier to process for motion/changes than blurred ones, so certain spatial frequencies can help highlight patterns in dynamic displays.
- Natural Visual Processing
 - The visual system is tuned to real-world patterns like horizontal/vertical references, subdued colors, and terrain/depth.
 - Bright colors in displays mimic natural attention cues (e.g., noticing bright flowers), and depth-based graphics align with everyday depth processing.
 - Implication: Visualizations should mimic real-world sensory experiences to reduce cognitive effort.

- Challenges in Visualization
 - Context and Bias: Interpretation depends on the user's background. Recent experiences can bias perception (e.g., seeing clusters where none exist if clusters were recently relevant).
 - Individual Differences: Past experiences shape how users interpret visuals, which may differ from the designer's intent.

Hierarchical Visualization: Cone Tree and Perspective Wall

Cone Tree

- Hierarchical data (e.g., organizational structures or document clusters) can be visualized using tree structures.
- However, traditional 2D trees become cluttered with large datasets, so 3D representations are used.

Description:

- This is a 3D visualization from the Information Visualizer (Xerox PARC).
- The tree's root node is at the apex, with child nodes arranged in a circular base.
- Each child node can be a parent to another cone, forming a nested structure.
- Selecting a node rotates it to the front for focus.

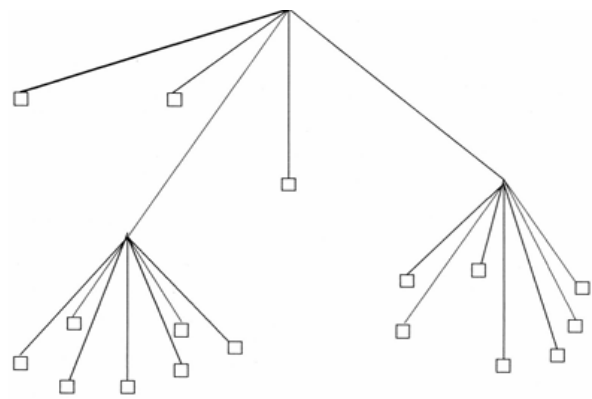


Figure 8.4 Cone Tree

Purpose: It shows the size and structure of subtrees, helping users understand the hierarchy and navigate to documents (represented as squares at the leaf nodes). Higher nodes may represent clusters or semantic centroids.

Example Use: In a document retrieval system, this could represent clusters of related articles, with leaf nodes as individual documents.

Perspective Wall

Description:

- The Information Visualizer, this divides information into three areas:
- a focused central area (e.g., "A GUI: The What Frame")
- two out-of-focus side areas (e.g., "Report," "Letter").
- The wall-like structure provides a perspective view.

Purpose: It allows users to focus on a specific area while keeping the broader context visible, aiding navigation in large datasets.

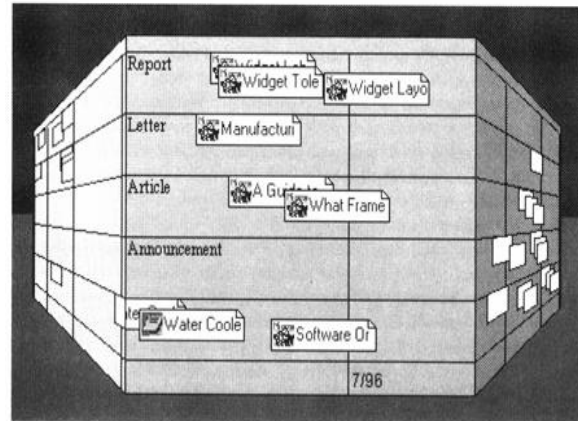


Figure 8.5 Perspective Wall
From inXight web site - www.inxight.com

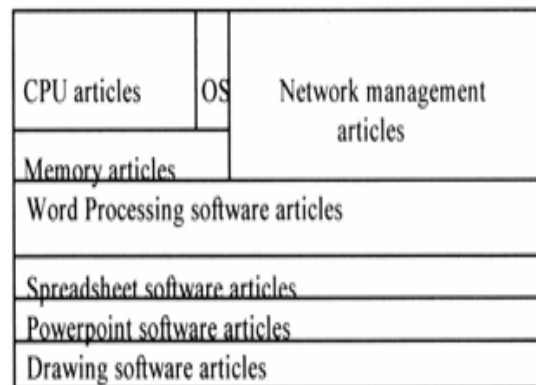
Example Use: In an IR system, this could display search results with the central area showing a selected document cluster, while side areas show related clusters.

Tree Maps for Hierarchical Data

Tree maps use nested rectangular boxes to represent hierarchical data, maximizing screen space.

In this example, the map shows computer-related articles divided into categories like CPU, OS, Memory, Network Management, and others (e.g., Word Processing, Spreadsheet Software).

Purpose: The size of each box reflects the number of items in a category, and the layout indicates relationships (e.g., CPU and OS are grouped under operating systems, separate from applications).



Example Use: A user searching for computer articles can quickly see which topics (e.g., OS) have the most documents and explore related areas.

Search Statement Analysis: Understanding Query Impact

Query Window: Displays and allows editing of the search query.

Graphic View Window: A scatterplot where each icon (circle or ellipse) represents a document or cluster. Circles show single items with relevance weights, while ellipses indicate clusters.

Item Summary Window: Shows bibliographic details of selected items.

Purpose: It helps users visualize the distribution of search results and their relevance, making it easier to refine queries.

Example Use: A user searching for “machine learning” can see clusters of documents and their relevance scores, then adjust the query based on the scatterplot.

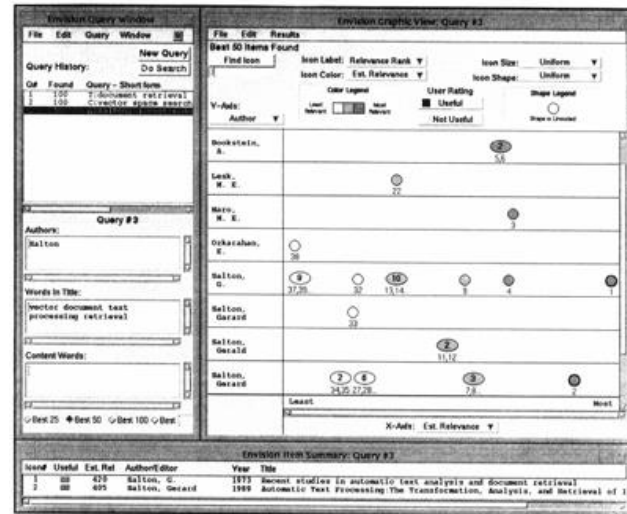


Figure 8.7 Envision Interface
(from SIGIR 96, page 68)

Visualization of Results

From SIGIR '96, this shows a grid where columns represent documents and rows represent query terms (e.g., “affirmative-action,” “construct-industry”).

The height of bars indicates the weight of each term in a document.

Purpose: Users can see which terms most influenced retrieval (by scanning columns) and how terms contributed overall (by scanning rows), helping identify underperforming terms for query refinement.

Example Use: For the query “How affirmative action affected the construction industry,” a user might notice “construct” has low impact and adjust the query to include related terms.

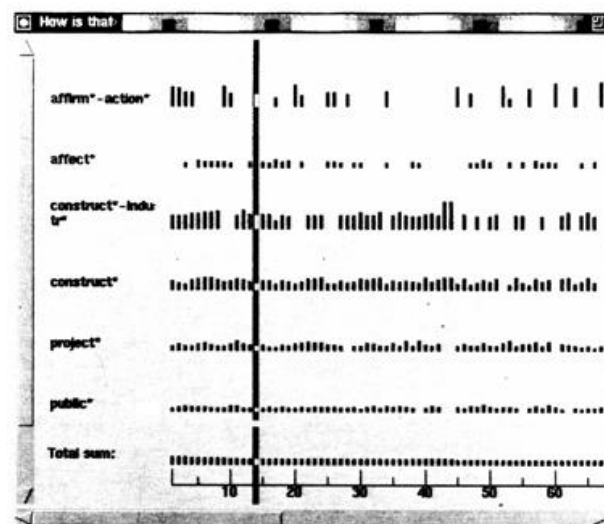


Figure 8.8 Visualization of Results
(from SIGIR 96, page 88)

DCARS Query Histogram

From Calspan's DCARS system, this histogram displays documents as rows, with tile bars showing the contribution of query terms (e.g., "arms," "proliferation") to retrieval.

The width of each bar reflects term importance.

From Calspan's DCARS system, this histogram displays documents as rows, with tile bars showing the contribution of query terms (e.g., "arms," "proliferation") to retrieval.

The width of each bar reflects term importance.

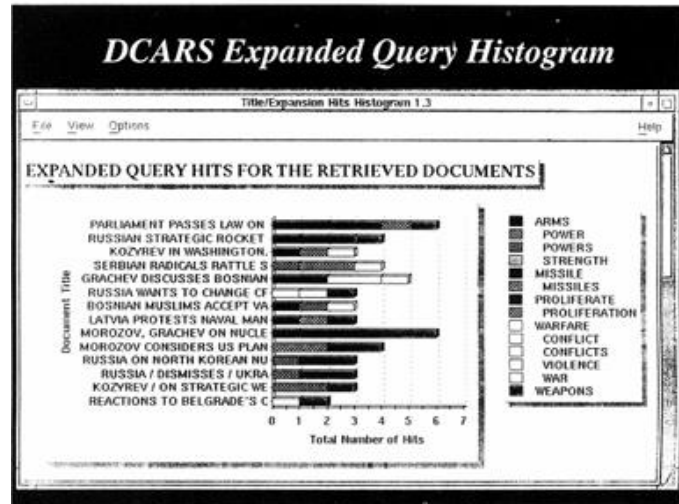


Figure 8.9 Example of DCARS Query Histogram (from briefing by CALSPAN)

Cityscape for Thematic Representation

This uses a 3D "city" metaphor where skyscrapers represent themes or concepts (e.g., document clusters).

The height of buildings indicates the importance or size of the theme, and connections between buildings show relationships.

Purpose: It allows users to "move" through the cityscape, zooming in on specific themes to reveal more details, providing an intuitive way to explore complex data.

Example Use: In a news retrieval system, a user might see a tall building for "oil spill" with connected buildings for related events, navigating to explore specific articles.

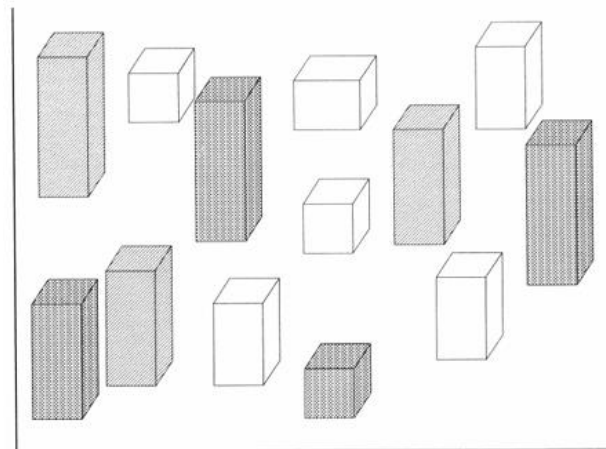


Figure 8.10 CityScape Example